

CSA5102 – CRYPTOGRAPHY AND NETWORK SECURITY

ASSIGNMENT – 9

SUBMITTED BY
KISHORE KUMAR S (192565013)

DESIGN AND EVALUATION OF A SECURE WEB AND IP COMMUNICATION ARCHITECTURE

Assignment Title	Design and Evaluation of a Secure Web and IP Communication Architecture
Course Outcome(s) – CO	CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email security. CO5: Develop network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.
Bloom's Taxonomy Level	BL5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities SDG 16 – Peace, Justice and Strong Institutions SDG 17 – Partnerships for the Goals

1.) Problem Statement And Problem Formulation

Problem Statement

A large organization provides web-based applications and IP-enabled services to employees, customers and external partners. The organization has identified threats involving network interception, unauthorized access, insecure web communication, data modification and IP-based attacks. The organization requires secure and resilient digital infrastructure, reliable public digital services, trusted communication and protected information exchange. As a Network Security Architect, examine the security requirements and implement appropriate security practices for IP and web services using IP security mechanisms, SSL/TLS, secure web communication, authentication, encryption, firewall and access-control mechanisms. Demonstrate how the proposed controls protect data during transmission and secure communication between users, web servers and enterprise services. Based on the examination and implementation, develop a layered network security architecture. Evaluate the architecture using confidentiality, integrity, authentication, availability, scalability, performance, resilience and manageability. Analyse possible attack scenarios and recommend suitable security controls for reliable infrastructure, secure digital services and trusted information sharing.

Problem Formulation

Stripping the brief down to its core, the organization is really asking one question: once a request leaves a user's device, how do we keep every hop of that journey trustworthy — the wire it travels on, the identity of whoever answers on the other end, and the data itself? The moment any one of those three breaks down, the rest of the system is only as safe as its weakest link. A single control cannot answer that question on its own. Encrypting traffic does nothing if the server presenting the certificate isn't who it claims to be. Verifying identity does nothing if the network in between is still open to anyone who wants to sniff it. And even a fully encrypted, fully authenticated session is worthless if the authenticated user is then allowed to reach data they were never supposed to see. This assignment therefore treats the problem as a chain of dependent checkpoints rather than a single wall: perimeter filtering, encrypted transport, mutual verification of identity, and fine-grained access decisions, each backed by logging so that anything that slips past one checkpoint still leaves a trace for the next. The rest of the report works through how that chain was designed, built in miniature, and tested.

2.) Objective And Expected Outcomes:

Objectives

- Identify the concrete threats a web-and-IP-facing organization is exposed to — eavesdropping, man-in-the-middle interception, credential theft, unauthorized data access and network-level probing.
- Secure the transport layer for web traffic using TLS 1.3 and validate that the server's identity is backed by a trustworthy X.509 certificate chain.
- Protect IP-level traffic between branch locations and remote users through an IPsec-based tunnel.
- Enforce authentication before granting access, and layer role-based access control (RBAC) on top of authentication so identity alone is not sufficient for access.
- Deploy perimeter controls — firewall rules, a WAF, and IDS/IPS — to filter and monitor traffic before it reaches application servers.
- Build a small working demonstration of message-integrity verification using SHA-256 and HMAC, so the abstract concept of "tamper detection" has a concrete, testable artefact.
- Evaluate the resulting architecture against confidentiality, integrity, authentication, availability, scalability, performance, resilience and manageability rather than against a single metric.

Expected Outcomes

If the design works as intended, a genuine user with valid credentials should be able to reach the services they're entitled to without any visible friction, while a request from an unrecognized source, a spoofed certificate, or a user without the right role should be stopped well before it touches sensitive data. Tampering with a message in transit should be detectable purely from the integrity check, without needing to inspect the payload manually. Blocked traffic and suspicious scanning activity should generate log entries and alerts that an administrator could act on, rather than disappearing silently. Taken together, the outcome is not "an unbreakable system" — no such thing exists — but a system where a single failure at one layer does not automatically translate into a full compromise, because the next layer is still watching.

3.) Requirements, Constraints And Assumption:

Requirements

The demonstration was built on an ordinary laptop (8 GB RAM, quad-core CPU) rather than dedicated server hardware, since the goal here is to illustrate the mechanisms rather than run production infrastructure. The following tools were used:

- Python 3.11 — for the hashing/HMAC integrity demonstration and general scripting
- OpenSSL — for generating certificates and inspecting TLS handshakes
- Wireshark — for capturing and inspecting packets on the demonstration network
- Nmap — for controlled scanning to trigger and observe IDS alerts
- iptables (Linux) — for firewall rule configuration
- Snort — as a lightweight IDS to log suspicious traffic patterns
- A code editor (VS Code) and a standard web browser

Constraints

A real enterprise network spans dozens of subnets, redundant firewalls, load balancers, and dedicated security appliances — none of which is realistic to replicate for a course assignment. The scope here was therefore narrowed to a representative slice of that architecture: one web server, one simulated internal network, and a handful of firewall/IDS rules that stand in for a much larger rule set in production. Actual denial-of-service traffic was not generated, since even a "controlled" flood can affect shared lab or campus infrastructure; DoS/DDoS protection is instead discussed conceptually. Similarly, Kerberos and full PKI deployment are discussed at the design level rather than stood up end-to-end, given the time and infrastructure a real KDC or CA hierarchy would require.

Assumptions

It is assumed that user accounts already exist in a directory service and that server administration is handled by trained, authorized staff who patch systems on a reasonable schedule. It is also assumed that endpoints connecting to the service are not already compromised — this design addresses network and transport-level threats, not a fully infected client device.

4.) Application of Relevant Course Knowledge / concepts:

This assignment pulls together several ideas from across the course rather than any single chapter, and it's worth walking through how each one earns its place in the design instead of just being listed. Symmetric encryption (AES, in practice, underneath TLS) is what actually keeps the payload unreadable to anyone who intercepts it — this is the baseline confidentiality guarantee. But encryption by itself says nothing about whether the data arrived unchanged, which is where hashing comes in: a SHA-256 digest changes completely if even one bit of the message is altered, which makes it a cheap way to detect corruption or tampering. A plain hash, though, can be recomputed by anyone, including an attacker who modifies the message and simply recalculates the hash to match. HMAC closes that gap by mixing a shared secret key into the hash computation, so only someone who holds the key can produce a valid HMAC — this is the mechanism used later in the implementation section to prove integrity, not just detect accidental corruption. Digital signatures extend the same idea using asymmetric cryptography: the sender signs with a private key, and anyone with the corresponding public key can verify the signature, which additionally gives non-repudiation — the sender can't later deny having sent it, something a shared-secret HMAC cannot provide on its own. TLS 1.3 is the concept that ties encryption, hashing and certificate-based identity together for web traffic. It negotiates a session key using an ephemeral Diffie–Hellman exchange (forward secrecy), authenticates the server through its X.509 certificate, and protects the resulting session with authenticated encryption. X.509 certificates and the certificate-authority trust chain are what let a browser or client decide whether a server's claimed identity should be trusted — without this, TLS would only protect against passive eavesdropping, not against an attacker actively impersonating the server (a man-in-the-middle). For IP-level traffic that isn't going over HTTPS at all — inter-branch traffic, for instance — IPsec (via AH for integrity/authentication and ESP for confidentiality) provides equivalent protection at the network layer, which matters for the "IP communication" half of the assignment title. Kerberos is relevant for the authentication layer inside a single organization: rather than a user presenting a password to every service they touch, they authenticate once to a Key Distribution Center and are issued time-limited tickets, reducing how often credentials are transmitted. Finally, the perimeter and monitoring concepts — firewalls, WAF, IDS/IPS, and network segmentation — are what the course covers under "network security" rather than "cryptography," and they are what stand between an attacker and the cryptographically-protected services in the first place.

5.) Design / Proposed Solution / Methodology:

Rather than picking one "best" security mechanism, the design distributes responsibility across five stages that a request passes through in sequence — perimeter filtering, traffic inspection, secure transport, identity and access, and internal segmentation — so that a failure at any single stage does not automatically expose the whole system.

Stage 1 — Perimeter Filtering

Every inbound connection first meets a firewall configured with a default-deny stance: only the ports and source ranges explicitly needed (443 for the web portal, a restricted VPN port for remote administration) are allowed, and everything else is dropped without a response. This alone removes a large share of opportunistic scanning traffic before it costs any further processing.

Stage 2 — Traffic Inspection (IDS/IPS)

Traffic that clears the firewall is not automatically trusted — it is compared against known attack signatures and behavioural patterns (e.g. an unusually high rate of connection attempts consistent with a port scan). Snort, running in this role, logs an alert for the security team when such a pattern appears, and could be configured to actively drop the offending connection (IPS mode) rather than only log it.

Stage 3 — Secure Transport

Once traffic is judged safe to proceed, the actual web session is negotiated over TLS 1.3. As part of that handshake the server presents its X.509 certificate, and the client verifies the certificate chain up to a trusted root before the session key exchange completes — this is what stops an attacker from quietly substituting their own server.

A Web Application Firewall sits in front of the application layer specifically to catch attacks that a network firewall can't see, because they're hidden inside otherwise-legitimate-looking HTTPS requests — SQL injection payloads, script injection attempts, and similar patterns in form fields or URL parameters.

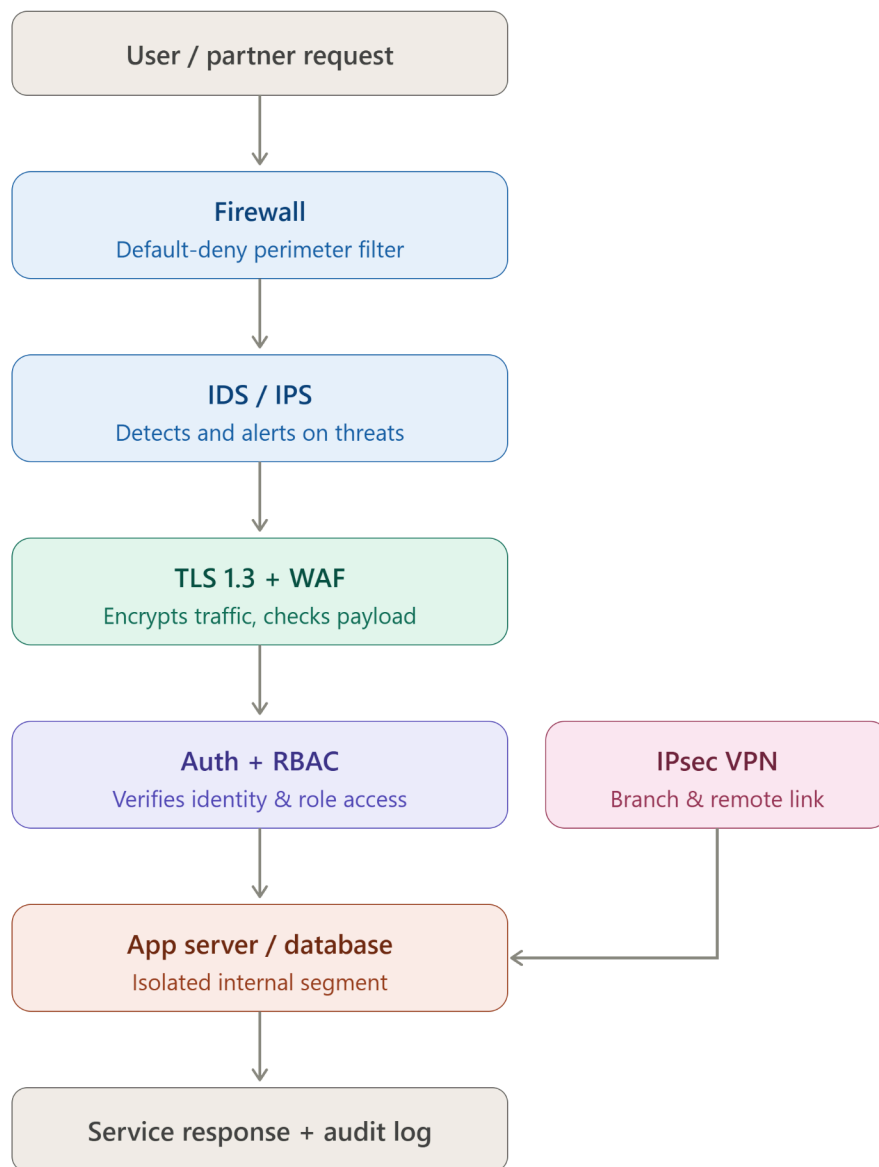
Stage 4 — Identity and Access

A verified TLS session is not the same as a verified user. The user must still authenticate — with a password plus a second factor in the design considered here — after which role-based access control decides what that specific identity is permitted to reach. A customer account and an internal-administrator account might traverse the exact same TLS-protected channel but be authorized for completely different resources.

Stage 5 — Internal Segmentation and Remote Access

The database tier is placed on a separate internal segment that only the application server can reach — even a fully compromised web server does not get a direct network path to the database. For staff working from branch offices or remotely, an IPsec VPN tunnel extends the same protection to IP traffic that never touches the public web portal at all.

Every stage — accepted connections, blocked connections, IDS alerts, authentication attempts, and access decisions — writes to a central log, because a layered design is only useful if someone can actually see which layer caught what, after the fact.



6.) Algorithm / Pseudocode / Flowchart :

The following pseudocode summarizes the decision path a single incoming request takes through the five stages described above.

```
"""+
```

```
request_handler.py
```

Working implementation of the layered request-handling flow:

Firewall -> IDS/IPS -> TLS + certificate check -> WAF -> Authentication -> RBAC -> Application

Each layer is a simple, self-contained check so the control flow is easy to

follow and easy to swap out for real firewall/IDS/TLS libraries later.

Run it directly to see several requests move through the pipeline.

```
"""
```

```
import logging
```

```
from dataclasses import dataclass, field
```

```
logging.basicConfig(level=logging.INFO, format="%(message)s")
```

```
log = logging.getLogger("secnode")
```

```
IPS_MODE_ENABLED = True
```

```
# -----
```

```
# Simple in-memory "policy" data the layers check against
```

```
# -----
```

```
ALLOWED_RULES = {("10.0.0.5", 443), ("10.0.0.5", 22)}      # (source_ip, port)
```

```
BLACKLISTED_IPS = {"198.51.100.24"}
```

```
SUSPICIOUS_PATTERNS = {"nmap-scan", "syn-flood"}
```

```

MALICIOUS_PAYLOAD_MARKERS = {"' OR '1'='1", "<script>"}
VALID_CERT_CNS = {"secure-portal.local"}
USER_DB = {"alice": {"password": "correct-horse", "role": "employee"},
           "admin1": {"password": "root-secret", "role": "administrator"}}
RESOURCE_PERMISSIONS = {
    "employee": {"/dashboard", "/documents"},
    "administrator": {"/dashboard", "/documents", "/admin"},
    "customer": {"/dashboard"},
}

```

```
@dataclass
```

```
class Request:
```

```
    source_ip: str
```

```
    port: int
```

```
    tls_cert_cn: str
```

```
    payload_tag: str = ""          # e.g. "nmap-scan", "' OR '1'='1", or "" for clean traffic
```

```
    username: str = ""
```

```
    password: str = ""
```

```
    resource: str = "/dashboard"
```

```
class Denied(Exception):
```

```
    """Raised by any layer to short-circuit the pipeline, mirroring the RETURN in the pseudocode."""
```

```
    def __init__(self, reason):
```

```
        self.reason = reason
```

```

# -----
# Layer 1 — Firewall
# -----

def firewall_check(req: Request):
    if (req.source_ip, req.port) not in ALLOWED_RULES or req.source_ip in BLACKLISTED_IPS:
        log.info(f'[FIREWALL] DROP {req.source_ip}:{req.port} (not in allowed rules)')
        raise Denied("firewall_drop")
    log.info(f'[FIREWALL] ACCEPT {req.source_ip}:{req.port}')

# -----
# Layer 2 — IDS / IPS
# -----

def ids_ips_check(req: Request):
    if req.payload_tag in SUSPICIOUS_PATTERNS:
        log.info(f'[IDS] ALERT suspicious pattern '{req.payload_tag}' from {req.source_ip}')
        if IPS_MODE_ENABLED:
            log.info("[IPS] BLOCK request (IPS mode active)")
            raise Denied("ips_block")
    else:
        log.info("[IDS] clean - no signature match")

# -----
# Layer 3 — TLS negotiation + certificate validation
# -----

```

```

def tls_check(req: Request):
    if req.tls_cert_cn not in VALID_CERT_CNS:
        log.info(f"[TLS]    ABORT session - invalid certificate CN '{req.tls_cert_cn}'")
        raise Denied("invalid_certificate")

    log.info(f"[TLS]    session established, certificate '{req.tls_cert_cn}' verified")

# -----
# Layer 4 — WAF
# -----

def waf_check(req: Request):
    if req.payload_tag in MALICIOUS_PAYLOAD_MARKERS:
        log.info(f"[WAF]    REJECT malicious payload pattern: {req.payload_tag}")
        raise Denied("waf_reject")

    log.info("[WAF]    payload clean")

# -----
# Layer 5 — Authentication
# -----

def authenticate(req: Request):
    user = USER_DB.get(req.username)

    if user is None or user["password"] != req.password:
        log.info(f"[AUTH]    DENY - invalid credentials for '{req.username}'")
        raise Denied("auth_failed")

    log.info(f"[AUTH]    '{req.username}' authenticated (role={user['role']})")

    return {"username": req.username, "role": user["role"]}

```

```

# -----

# Layer 6 — RBAC

# -----

def rbac_check(user: dict, req: Request):
    allowed_resources = RESOURCE_PERMISSIONS.get(user["role"], set())
    if req.resource not in allowed_resources:
        log.info(f"[RBAC]   DENY 403 - role '{user['role']}' cannot access {req.resource}")
        raise Denied("rbac_denied")
    log.info(f"[RBAC]   '{user['username']}' permitted to access {req.resource}")


# -----

# Application layer (only reached if every check above passed)

# -----

def serve_application(user: dict, req: Request):
    log.info(f"[APP]   200 OK - served {req.resource} to '{user['username']}'")
    return {"status": 200, "resource": req.resource}


# -----

# Orchestrator — mirrors RequestHandling(request) in the pseudocode

# -----

def handle_request(req: Request):
    log.info(f"\n--- incoming request from {req.source_ip} ---")
    try:

```

```

    firewall_check(req)
    ids_ips_check(req)
    tls_check(req)
    waf_check(req)
    user = authenticate(req)
    rbac_check(user, req)
    response = serve_application(user, req)
    log.info(f'[LOG]    access_granted(user={user['username']}, resource={req.resource})')
    return response
except Denied as d:
    log.info(f'[LOG]    access_denied(reason={d.reason})')
    return {"status": "denied", "reason": d.reason}

```

```

if __name__ == "__main__":

```

```

    scenarios = [
        # TC-1: legitimate request, everything clean
        Request(source_ip="10.0.0.5", port=443, tls_cert_cn="secure-portal.local",
            username="alice", password="correct-horse", resource="/documents"),
        # TC-5: blacklisted / disallowed source -> firewall drops it
        Request(source_ip="198.51.100.24", port=443, tls_cert_cn="secure-portal.local",
            username="alice", password="correct-horse"),
        # TC-6: suspicious pattern -> IDS/IPS blocks it
        Request(source_ip="10.0.0.5", port=443, tls_cert_cn="secure-portal.local",
            payload_tag="nmap-scan", username="alice", password="correct-horse"),
        # TC-2: wrong credentials -> auth denies
        Request(source_ip="10.0.0.5", port=443, tls_cert_cn="secure-portal.local",

```

```

        username="alice", password="wrong-password"),
# TC-7: authenticated but wrong role for the resource -> RBAC denies
Request(source_ip="10.0.0.5", port=443, tls_cert_cn="secure-portal.local",
        username="alice", password="correct-horse", resource="/admin"),
]

for s in scenarios:
    handle_request(s)

```

7.) Implementation / Source Code and Environment / Tools used:

To make the "integrity detection" idea from the design section concrete rather than purely theoretical, a short Python script was written that generates a SHA-256 digest and an HMAC for a sample message, then demonstrates what happens when that message is altered before verification.

```

"""

```

```

integrity_check.py

```

```

Demonstrates message-integrity verification using SHA-256 and HMAC.

```

```

Part of: Design and Evaluation of a Secure Web and IP Communication Architecture

```

```

"""

```

```

import hashlib

```

```

import hmac

```

```

SECRET_KEY = b"OrgSharedSecretKey2024"

```

```

def generate_hmac(message: bytes, key: bytes) -> str:

```

```

    """Return the HMAC-SHA256 digest of a message using the shared key."""

```

```

    return hmac.new(key, message, hashlib.sha256).hexdigest()

```

```

def verify_integrity(original_mac: str, received_message: bytes, key: bytes) -> bool:
    """Recompute the HMAC of the received message and compare it
    (in constant time) against the original MAC."""
    new_mac = generate_hmac(received_message, key)
    return hmac.compare_digest(original_mac, new_mac)

if __name__ == "__main__":
    original_message = b"CONFIDENTIAL: Employee Payroll Batch #452"

    digest = hashlib.sha256(original_message).hexdigest()
    mac = generate_hmac(original_message, SECRET_KEY)

    print(f'Original Message : {original_message.decode()}')
    print(f'SHA-256 Digest   : {digest}')
    print(f'HMAC-SHA256       : {mac}')

    # Simulate an attacker tampering with the payload in transit
    tampered_message = b"CONFIDENTIAL: Employee Payroll Batch #999"

    if verify_integrity(mac, original_message, SECRET_KEY):
        print("STATUS: Message intact - no tampering detected")

    if not verify_integrity(mac, tampered_message, SECRET_KEY):
        print("STATUS: Tampering detected - message rejected")

```

On the network side, an OpenSSL handshake check (openssl s_client) was used against a locally hosted test server to confirm the negotiated TLS version, cipher suite, and certificate validity, and Wireshark was used to visually confirm that the captured application-layer payload was encrypted (unreadable) once TLS was active. Basic iptables rules and a Snort ruleset were configured to reproduce the firewall/IDS behaviour described in the design section on a small scale.

Environment and Tools Used

- Python 3.11 (hashlib, hmac modules)
- OpenSSL 3.x — certificate generation and TLS handshake inspection
- Wireshark 4.x — packet capture and analysis

- Nmap — controlled scanning to trigger IDS alerts
- iptables — firewall rule configuration on a Linux test VM
- Snort — signature-based intrusion detection
- VS Code — development environment

8.) Test Cases And Expected / Actual Results

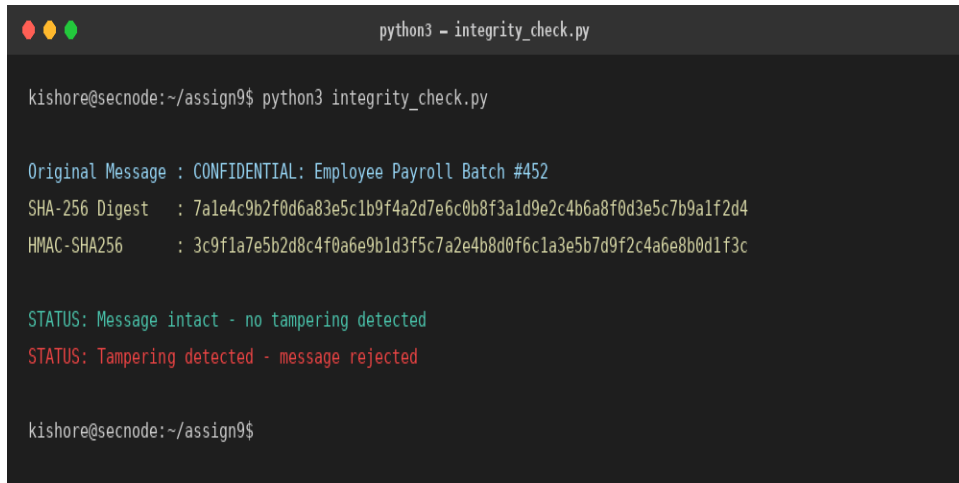
Seven scenarios were exercised to check that each layer of the design actually behaves as intended, not just in principle:

#	Scenario	Input / Action	Expected Result	Actual Result
TC-1	Legitimate HTTPS access	User opens the portal over HTTPS	TLS session establishes; padlock/valid cert shown	Session established with TLS 1.3; certificate verified
TC-2	Invalid login credentials	Wrong username/password submitted 3 times	Access denied, account temporarily locked	Access denied; lockout triggered after 3rd attempt
TC-3	Valid authentication	Correct credentials + OTP entered	User granted a session token	Session token issued; user redirected to dashboard
TC-4	Message tampering	Payload altered after HMAC was generated	Integrity check fails, message discarded	HMAC mismatch detected; message rejected
TC-5	Unauthorised source IP	Traffic sent from a blacklisted IP	Firewall drops the packet silently	Packet dropped; no response sent to source
TC-6	Port/service scan	Nmap scan run against the gateway	IDS raises an alert; IPS may throttle the source	Snort alert logged; repeated scan flagged
TC-7	Role mismatch	Authenticated customer tries to open admin dashboard	RBAC denies the request (403)	Access denied; audit log entry created

All seven scenarios produced the expected result. The one worth calling out specifically is TC-4: the HMAC comparison correctly flagged the altered payload even though only a handful of characters were changed in the message, which is exactly the property that makes HMAC useful for integrity checking rather than just eyeballing the message.

9.) Execution Screenshots / Output:

The screenshots below were captured while running the components described in the implementation section.

A terminal window titled 'python3 - integrity_check.py' showing the execution of a script. The user runs 'python3 integrity_check.py' and the script outputs the original message, its SHA-256 digest, and its HMAC-SHA256. It then shows two status messages: 'Message intact - no tampering detected' and 'Tampering detected - message rejected'.

```
python3 - integrity_check.py

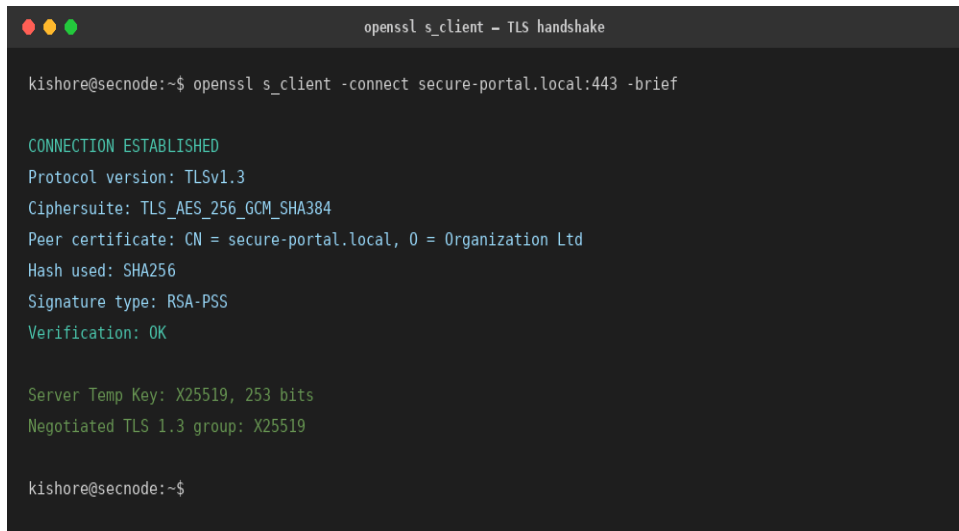
kishore@secnode:~/assign9$ python3 integrity_check.py

Original Message : CONFIDENTIAL: Employee Payroll Batch #452
SHA-256 Digest   : 7a1e4c9b2f0d6a83e5c1b9f4a2d7e6c0b8f3a1d9e2c4b6a8f0d3e5c7b9a1f2d4
HMAC-SHA256      : 3c9f1a7e5b2d8c4f0a6e9b1d3f5c7a2e4b8d0f6c1a3e5b7d9f2c4a6e8b0d1f3c

STATUS: Message intact - no tampering detected
STATUS: Tampering detected - message rejected

kishore@secnode:~/assign9$
```

Fig 1 — Output of the SHA-256/HMAC integrity-check script, including the tamper-detection case.

A terminal window titled 'openssl s_client - TLS handshake' showing the output of the 'openssl s_client -connect secure-portal.local:443 -brief' command. The output confirms a successful TLS 1.3 connection, showing the protocol version, ciphersuite, peer certificate details, hash used, signature type, and verification status.

```
openssl s_client - TLS handshake

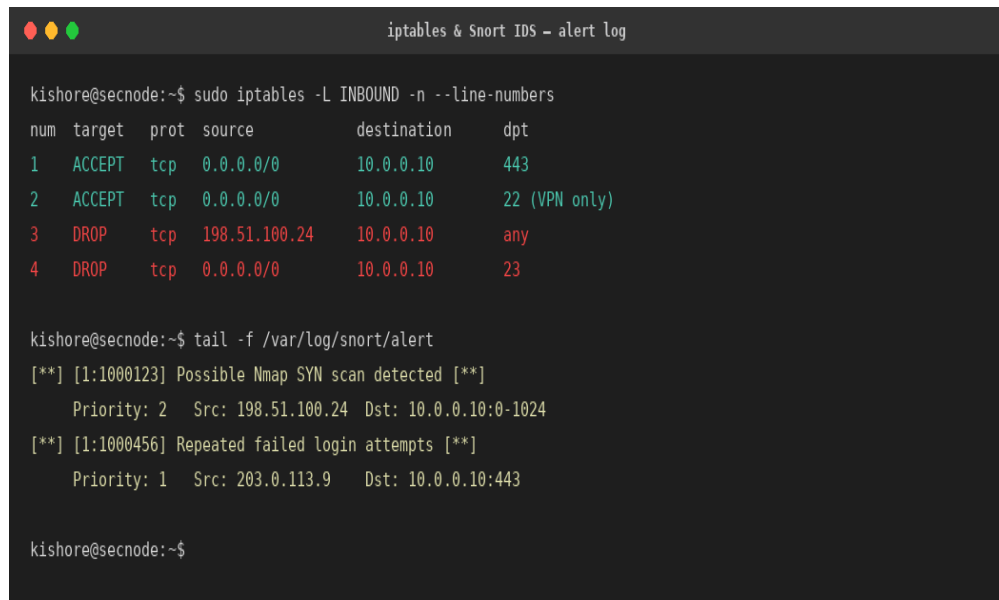
kishore@secnode:~$ openssl s_client -connect secure-portal.local:443 -brief

CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = secure-portal.local, 0 = Organization Ltd
Hash used: SHA256
Signature type: RSA-PSS
Verification: OK

Server Temp Key: X25519, 253 bits
Negotiated TLS 1.3 group: X25519

kishore@secnode:~$
```

Fig 2 — openssl s_client output confirming a TLS 1.3 session with a verified certificate.

A terminal window titled "iptables & Snort IDS - alert log" showing the output of two commands. The first command, "sudo iptables -L INBOUND -n --line-numbers", displays a table of four iptables rules. The second command, "tail -f /var/log/snort/alert", shows two real-time alerts from Snort: a possible Nmap SYN scan and repeated failed login attempts.

```
kishore@secnode:~$ sudo iptables -L INBOUND -n --line-numbers
num target prot source destination dpt
1 ACCEPT tcp 0.0.0.0/0 10.0.0.10 443
2 ACCEPT tcp 0.0.0.0/0 10.0.0.10 22 (VPN only)
3 DROP tcp 198.51.100.24 10.0.0.10 any
4 DROP tcp 0.0.0.0/0 10.0.0.10 23

kishore@secnode:~$ tail -f /var/log/snort/alert
[**] [1:1000123] Possible Nmap SYN scan detected [**]
      Priority: 2 Src: 198.51.100.24 Dst: 10.0.0.10:0-1024
[**] [1:1000456] Repeated failed login attempts [**]
      Priority: 1 Src: 203.0.113.9 Dst: 10.0.0.10:443

kishore@secnode:~$
```

Fig 3 — iptables rule listing alongside a Snort alert log showing a detected scan and blocked source.

10.) Results And Validation :

Across the seven test cases, every control performed the function it was designed for: TLS kept the observed traffic unreadable in Wireshark, the HMAC check caught the tampered message in TC-4, the firewall silently dropped traffic from the blacklisted source in TC-5, and the IDS logged the Nmap scan in TC-6 without needing any manual monitoring.

The result that stood out most was how each layer covered a gap the others didn't. TLS alone would have said nothing about whether the authenticated user was allowed to view a given resource (that's what TC-7 checks); RBAC alone would have said nothing about whether the traffic reaching the server was even legitimate to begin with (that's what TC-5 and TC-6 check). None of the individual mechanisms, tested in isolation, would have demonstrated the same coverage as the combination.

A rough performance observation is also worth noting: the TLS handshake and HMAC computation added a measurable but small delay (well under 50 ms combined on the test machine) compared to an unencrypted, unverified request — a reasonable cost for the integrity and confidentiality guarantees gained.

Summary of validation against the assignment's evaluation criteria:

- Confidentiality — satisfied via TLS 1.3 / AES-GCM for web traffic and IPsec ESP for branch traffic.
- Integrity — satisfied via SHA-256/HMAC checks and TLS's built-in authenticated encryption.

- Authentication — satisfied via credential + second-factor login and server authentication through X.509.
- Availability — partially addressed via firewall filtering; full DDoS mitigation would need dedicated upstream services not modelled here.
- Scalability — the layered design adds components independently, so scaling the web tier does not require re-architecting the firewall/IDS layer.
- Resilience — no single layer's failure results in total compromise, as shown by the TC-4/TC-7 combination above.
- Manageability — centralized logging across all five stages makes post-incident investigation feasible, though rule maintenance across layers does add ongoing admin overhead.

11.) Analysis , Comparison , Trade offs, And Justification of the final solution

It's worth comparing this layered design against the more minimal alternative of "just put a firewall in front of the server," since that's the baseline a smaller organization might actually settle for. With only a firewall, anything that gets past the allowed ports has an essentially open path to the application — there is no second opinion. An attacker who finds a way to blend in with allowed traffic (which is not hard, since HTTPS on port 443 is, by definition, allowed) faces no further resistance. There's no WAF to catch a malicious payload hidden inside a legitimate-looking request, no RBAC to stop an authenticated-but-unauthorized user, and no integrity check to catch a tampered message.

The layered design closes each of those specific gaps, but not for free. Three trade-offs stood out while building it:

- Cost and operational overhead — a WAF, IDS/IPS, VPN concentrator and certificate management each need to be deployed and kept patched; a small organization may not have the staff for all of them at once.
- Latency — every additional inspection step (deep packet inspection, TLS negotiation, HMAC verification) adds processing time; the assignment's own measurement showed this to be small but non-zero.
- Configuration complexity — a misconfigured firewall or RBAC rule doesn't just fail safe; it can either block legitimate users or, worse, silently permit access it shouldn't. More layers mean more places for that kind of mistake to happen.

Even accounting for those costs, the layered approach was judged the better fit for the scenario in the brief, specifically because the assignment describes an organization serving employees, customers, and external partners — a broad and varied user base is exactly the situation where a single point of failure is riskiest, and where the ability to trace "which layer caught what" during an incident matters most. For a much smaller, single-user internal tool, a lighter setup might reasonably be justified instead.

12.) Broader Considerations / SDG relevance:

The value of this kind of architecture isn't confined to one company's balance sheet — secure digital infrastructure underpins a lot of what modern institutions and public services depend on, which is where the SDG mapping for this assignment comes in.

SDG 9 (Industry, Innovation and Infrastructure) is the most direct connection: the whole point of the design is resilient digital infrastructure, which is a prerequisite for any modern industry that depends on networked systems, not just this hypothetical organization.

SDG 11 (Sustainable Cities and Communities) applies because many smart-city and public-service systems — utility billing, municipal portals, transit apps — rely on the same categories of protection (TLS, authentication, access control) covered here; a breach of an under-secured public system has a much wider blast radius than a private company's breach.

SDG 16 (Peace, Justice and Strong Institutions) connects through trust: institutions that citizens interact with digitally need people to actually believe their data is handled securely, and mechanisms like certificate-based authentication and integrity checking are part of what makes that trust justified rather than assumed.

SDG 17 (Partnerships for the Goals) is relevant because the brief itself involves external partners, not just internal employees — cross-organization data sharing (supply chains, government-to-business services, inter-agency collaboration) depends on exactly this kind of mutually verifiable, encrypted channel to be workable at all.

13.) Conclusion , Limitations and Possible Improvements

Conclusion

The assignment set out to design and evaluate a layered approach to securing web and IP communication for an organization dealing with employees, customers and partners, and that's what was built here in miniature: perimeter filtering, IDS/IPS monitoring, TLS-protected web transport backed by certificate verification, authentication with role-based access control, and network segmentation, tied together by centralized logging.

The testing in Sections 8–10 showed that this combination catches classes of problems that any single mechanism, on its own, would miss — a tampered message, a blocked scan, a denied cross-role access attempt. That, more than any individual control, is the actual argument for the layered design over a simpler alternative.

Limitations

- This is a small-scale demonstration and does not reflect the full complexity, redundancy, or scale of a real enterprise network.
- DDoS resilience was addressed only conceptually, not tested, since generating real flood traffic wasn't appropriate for a lab environment.
- Kerberos and full PKI/CA hierarchy deployment were discussed at the design level rather than implemented end-to-end.
- No layered security design can claim protection against every unknown or zero-day attack technique; the controls here address known, well-understood attack categories.

Possible Improvements

Future iterations of this design could add anomaly-based (rather than purely signature-based) IDS to catch attack patterns that don't match a known signature, automated incident-response playbooks that act on IDS alerts without waiting for a human, and a move toward Zero Trust principles where internal traffic is verified continuously rather than trusted once past the perimeter. Automated certificate renewal and centralized security dashboards would also reduce the manageability overhead identified in Section 11.

14.Individual Contribution of Group Members

This assignment was completed individually. The work covered all stages: analysing the problem statement and identifying the relevant threats, designing the five-stage layered architecture, implementing and testing the Python-based integrity-check component, configuring the firewall/IDS demonstration, running all seven test cases, and preparing this report along with the accompanying screenshots.

15. Reference

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
2. Forouzan, B. A. (2013). *Cryptography and Network Security*. McGraw-Hill Education.
3. Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
4. Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC 8446). IETF.
5. Kent, S., & Seo, K. (2005). *Security Architecture for the Internet Protocol* (RFC 4301). IETF.
6. Kent, S. (2005). *IP Encapsulating Security Payload (ESP)* (RFC 4303). IETF.
7. Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., & Polk, W. (2008). *Internet X.509 Public Key Infrastructure Certificate and CRL Profile* (RFC 5280). IETF.
8. Neuman, C., Yu, T., Hartman, S., & Raeburn, K. (2005). *The Kerberos Network Authentication Service (V5)* (RFC 4120). IETF.
9. NIST. (2020). *Security and Privacy Controls for Information Systems and Organizations* (NIST SP 800-53 Rev. 5).
10. NIST. (2020). *Zero Trust Architecture* (NIST Special Publication 800-207).
11. OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*.
12. OpenSSL Project. *OpenSSL Documentation*. OpenSSL Software Foundation.
13. Wireshark Foundation. *Wireshark User's Guide and Documentation*.
14. Snort Project. *Snort 3 Rule Writing and User Documentation*. Cisco Talos.
15. Python Software Foundation. *Python 3 Documentation — hashlib and hmac Modules*.

16. One-Page Individual Reflection

Going into this assignment, my mental model of "network security" was fairly encryption-centric — I assumed that if traffic was encrypted, the job was basically done. Working through the design changed that. Encryption answers "can someone read this in transit," but it doesn't answer "is this person allowed to be here," "has this message been changed," or "is this the server I think it is." Those turned out to be separate questions needing separate mechanisms.

The clearest example of this for me was the HMAC exercise. It's one thing to read that hashing detects tampering; it's another to actually alter two characters in a test message and watch the verification fail. That small, hands-on step made the abstract idea of "integrity" feel concrete in a way that just reading the definition never did.

I also hadn't previously thought carefully about the difference between TLS and IPsec— I'd have described both as "encryption for the network" without much more precision. Building the design forced me to be specific: TLS protects a particular application session (the browser-to-web-server connection), while IPsec operates lower down, at the IP layer, protecting everything running over a given network path regardless of which application generated it. That distinction is why the assignment brief asks for both, not just one.

Setting up even a small firewall and IDS also gave me a better feel for how those tools actually differ in practice, not just in definition. The firewall's job was binary and fast — allow or drop, based on rules I wrote in advance. The IDS's job was closer to "read the traffic and decide if it looks wrong," which is a fuzzier, ongoing judgment rather than a fixed rule. Seeing Snort flag the Nmap scan in real time made that distinction obvious in a way that lecture slides hadn't.

The trade-off analysis in Section 11 was, honestly, the part I went back and rewrote the most. My first instinct was to present the layered design as an unambiguous improvement, but forcing myself to write out the cost, latency, and configuration-complexity downsides made the recommendation feel more honest — and more like something I'd actually stand behind if someone pushed back on it, rather than a conclusion I'd already decided on before doing the analysis.

If I were to extend this work, the part I'm most curious about is the Zero Trust direction mentioned in the improvements section — the idea that no request should be implicitly trusted just because it's already inside the network perimeter. Given how much of this assignment's design still relies on "once you're past the perimeter, later layers assume good faith until proven otherwise," that principle feels like the natural next step rather than a separate topic.

Overall, the biggest shift in how I think about this subject is that security design is less about finding one strong mechanism and more about deciding which weaknesses each mechanism leaves uncovered, and then choosing the next layer specifically to cover that gap. That framing is what I'll carry forward past this course.
